DESENVOLVIMENTO FRONT-END II

SPRINT 3 — RELATÓRIO FINAL

Este é o documento final do trabalho. Ele acompanha a apresentação e a demo ao vivo. Registre aqui não só o que foi construído, mas o que o grupo aprendeu ao longo do semestre.

1. Identificação

Nome do Sistema	Estagiou
URL do Repositório	*https://github.com/henriquerodrigues-06/Estagiou*
URL da Demo (se houver)	-
Integrantes	Henrique Pereira Rodrigues Nunes, Moacyr Mendes de Oliveira Neto
Data da Apresentação	*17/11/2026*

2. Checklist de Entrega

Marque o que o grupo está entregando. O professor vai verificar cada item durante a apresentação.

Item	Status
Repositório com código completo e histórico de commits de todos os integrantes	X Home Sim ☐ Não
README com instruções de como instalar as dependências e rodar o projeto (npm install + npm run dev)	x Sim ☐ Não
Aplicação React rodando sem erros no navegador	x Sim ☐ Não
Múltiplas páginas com menu de navegação funcionando	x Sim ☐ Não
Consumo de API RESTful com CRUD completo (Create, Read, Update, Delete)	x Sim ☐ Não
Relatório Final (este documento) preenchido	x Sim ☐ Não
Demo ao vivo preparada para a apresentação	x Sim ☐ Não

3. O Que Foi Construído

3.1 Páginas e componentes entregues

#	Página / Componente	Tipo	Funciona na demo?

1	Home	X Página ☐ Componente	X Sim ☐ Não
2	Cadastrar	X Página ☐ Componente	X Sim ☐ Não
3	DetalhesVaga	X Página ☐ Componente	X Sim ☐ Não
4	Perfil	X Página ☐ Componente	X Sim ☐ Não
5	Card	☐ Página X Componente	X Sim ☐ Não
6	BarraNav	☐ Página X Componente	X Sim ☐ Não

3.2 Integração com o Back-end (CRUD RESTful)

Entidade gerenciada no CRUD	Vagas de Estágio (Estágios).
Endpoints consumidos	`GET /vagas`, `GET /vagas/listar/:id`, `GET /vagas/listar/termo`, `POST /vagas/cadastrar`, `PATCH /vagas/atualizar/:id` e `DELETE /vagas/deletar/:id`.
Como fez as requisições?	Utilizando a biblioteca Axios configurada numa instância base (`api.js`), com chamadas assíncronas (`async/await`) isoladas na camada de serviços (`vagasService.js`).
Tratamento de erros	Uso robusto da estrutura `try/catch/finally`. Gestão do estado de erro (`setErro`) para renderizar mensagens de falha amigáveis (usando `alert-danger` do Bootstrap) na interface caso o servidor caia, e gestão de um estado de carregamento (`setCarregando`) com o ícone *spinner* (`Loader2` do lucide-react) bloqueando interações indevidas.
Integração funcionando?	☑ Totalmente.

3.3 O que ficou fora do escopo e por quê

- **Funcionalidade de Edição do "Meu Perfil":** O botão de "Editar Perfil" foi incluído na interface visual (`Perfil.jsx`), mas atualmente emite apenas um alerta ("Funcionalidade será implementada em atualizações futuras").
- **Porquê:** Decidimos priorizar o funcionamento perfeito do fluxo central (CRUD de Vagas e integração com o MongoDB), e devido às restrições de tempo desta sprint, o sistema de edição e autenticação do aluno/empresa ficou para uma etapa futura.

4. Qualidade do Código React

4.1 Decisões técnicas e de organização

Organização do projeto	Arquitetura dividida por pastas de responsabilidade: `/src/pages` (telas principais), `/src/components` (fragmentos reaproveitáveis), `/src/services` (comunicação com a API) e `/src/assets` (imagens).
Gerenciamento de estado	Utilização dos hooks nativos `useState` (para dados locais e controle de loading/erro) e `useEffect` (para disparar as chamadas à API no momento da montagem dos componentes).
Roteamento	Implementado com `react-router-dom`, utilizando `BrowserRouter`, `<Routes>`, `<Route>` e hooks de navegação programática como o `useNavigate` e o `useParams` (para capturar IDs da URL).
Estilização	Mistura de classes utilitárias do *Bootstrap* para layouts e grid rápidos, combinada com *CSS Puro* (`index.css`) com variáveis nativas (ex: `--unicesusc-bordo`) para a paleta de cores da instituição.
Boas práticas adotadas	Separação estrita de responsabilidades (os componentes não sabem como o Axios funciona, apenas chamam o `vagasService.js`); criação de funções utilitárias internas (como o `normalizarVaga`) para conciliar a diferença entre o `_id` do MongoDB e a propriedade `id` usada no frontend; tratamento explícito de estados assíncronos (pending, fulfilled, rejected).

5. Controle de Versão

Total de commits no projeto	14
Organização dos arquivos no repositório	*src/assets (imagens), /src/components (elementos visuais reutilizáveis), /src/pages (telas roteáveis) e /src/services (lógica de banco de dados local).*
Todos os integrantes commitaram?	*X Sim ☐ Não (explique: _____)*

6. Reflexão do Grupo

6.1 O que o grupo aprendeu na prática

Aprendemos profundamente como integrar um frontend e um backend reais, substituindo a facilidade do `localStorage` pela complexidade da rede. Ficou muito clara a importância de tratar a "demora" da rede usando *Promises* (`async/await`) e de gerenciar estados intermediários de carregamento (*loading*) e erro. Percebemos que o React quebra muito fácil se não garantirmos

que a variável que guarda os dados tem um valor inicial antes do backend responder (como iniciar com `null` ou um array vazio `[]`).

6.2 O que faria diferente

Na verdade, não mudaríamos muita coisa. O processo de desenvolvimento fluiu de forma bem tranquila, e a base estrutural que definimos no início do projeto se mostrou muito sólida. A escolha das tecnologias (React, Node.js e MongoDB) e a organização da arquitetura — com a clara separação entre rotas, controllers e serviços — facilitaram bastante a integração final.

6.3 O projeto tem futuro?

Sim, muito potencial. A dor de conectar especificamente os alunos do Unicesusc às empresas locais de forma direta é real. Para levar o Estagiou ao próximo nível, precisaríamos de: 1) Fazer o *deploy* da aplicação na nuvem; 2) Criar um sistema de autenticação (Login/JWT) para separar a visão da "Empresa" (que publica as vagas) da visão do "Aluno" (que só as vê e se candidata); 3) Adicionar um painel para a empresa poder gerenciar as candidaturas recebidas dentro da própria plataforma em vez de apenas ser redirecionada.

Bom trabalho ao longo do semestre.